

Experiment No.	1
Experiment Title	Data Preprocessing and Exploratory Data Analysis ¹
Student Name	Samyuktha V
Register Number	3122247001053
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	12/08/26

1 Objective

To load a real-world dataset, perform data preprocessing (missing value handling, duplicate removal, feature scaling), carry out exploratory data analysis (EDA) with appropriate visualizations, and prepare the data (train/test split) for downstream Machine Learning experiments – all wrapped in a single, generic, reusable function that works on any tabular classification dataset.

2 Problem Statement

Given a raw CSV dataset containing numerical features describing an email (word frequencies, character frequencies, capital-run-length statistics) and a target label indicating whether the email is spam or not, clean and prepare the dataset so it is ready for classification modelling. Input: raw dataframe + target column name. Output: a cleaned, scaled, and split dataset (`X_train`, `X_test`, `y_train`, `y_test`) along with EDA visualizations.

3 Dataset Description

Dataset Name	Spambase (updated)
Dataset Source	UCI Machine Learning Repository (Spam-base)
Number of Samples	4,353 (before cleaning); 3,968 after removing 384 duplicate rows and 1 row with a missing target value
Number of Features	57 numeric features (word frequencies, character frequencies, capital-run-length statistics)
Number of Classes	2 (0 = Ham, 1 = Spam)
Missing Values	25 columns had exactly 1 missing value each (including the target column <code>spam</code>); the row with the missing target was dropped, remaining numeric gaps were filled with the column median
Train-Test Split	80% train (3,174 rows) / 20% test (794 rows) – stratified on the target, <code>random_state=42</code>

4 Exploratory Data Analysis

- Dataset overview (`df.head()`, `df.info()`)
- Statistical summary (`df.describe()`)
- Missing value analysis
- Class distribution
- Correlation matrix
- Feature distribution (histograms, boxplots)

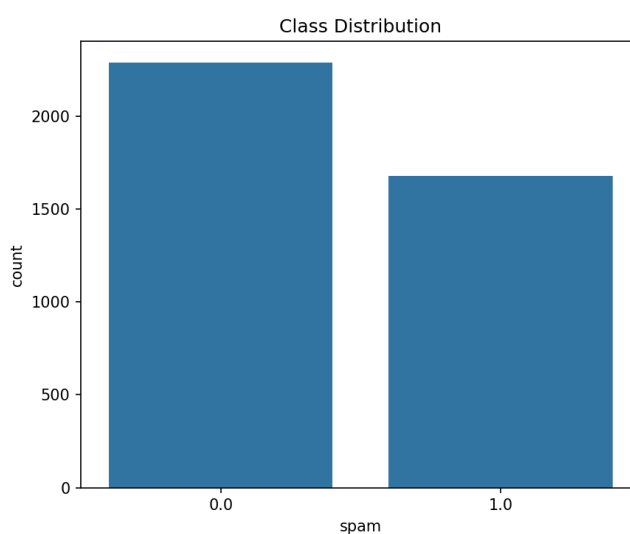


Figure 1: Class distribution of the target variable (spam vs. ham)

Inference:

After cleaning, the training set has 1,831 ham emails (class 0) and 1,343 spam emails (class 1) – roughly a 58:42 split. This is mildly imbalanced but not severe enough to require resampling techniques; it does mean plain accuracy alone can be slightly misleading, so precision/recall/F1 are also worth tracking for this dataset, which is exactly what the Experiment 2 evaluation pipeline reports.

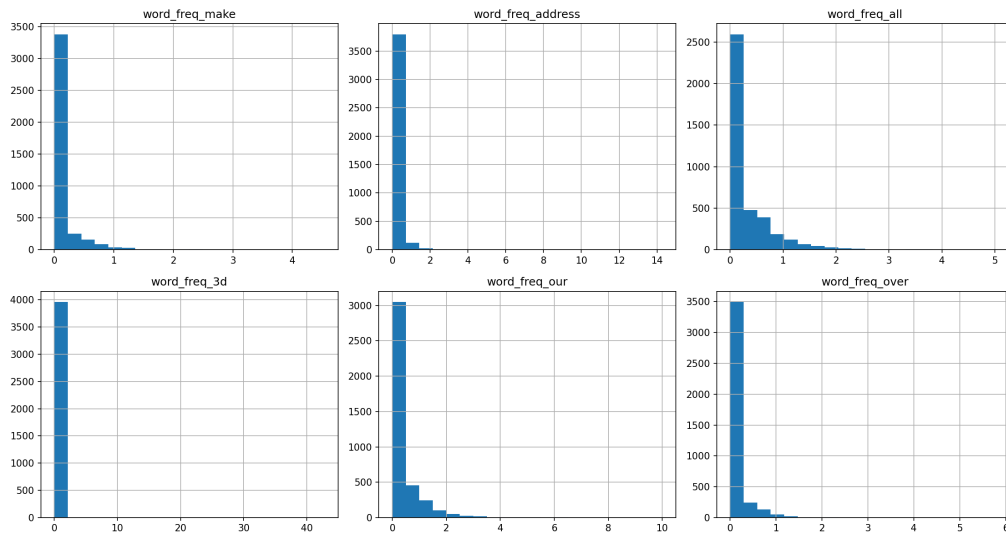


Figure 2: Distribution of the first 6 numeric features

Inference:

Most word-frequency features are heavily right-skewed, with the bulk of values at or near zero and a long tail of larger values – expected for word-count-style data, since most emails don’t contain most given words at all. This skew is one of the reasons Gaussian Naive Bayes (which assumes normally distributed features) is a theoretically poor fit for this data compared to Multinomial Naive Bayes, a comparison carried out directly in Experiment 2.

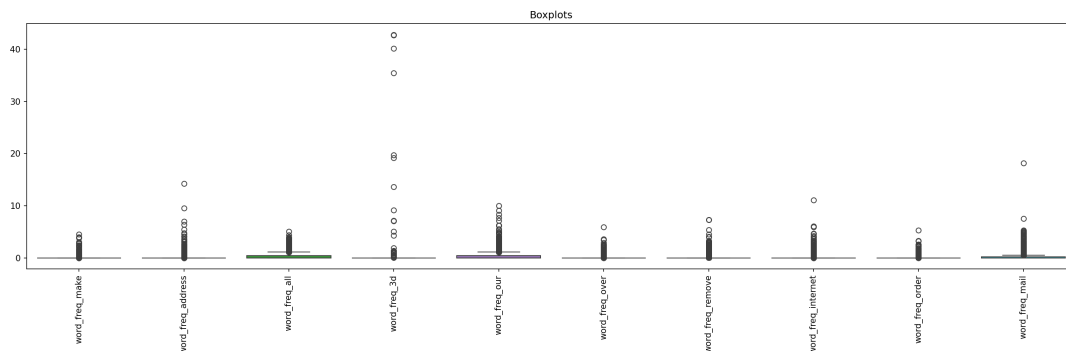


Figure 3: Boxplots of the first 10 numeric features

Inference:

The boxplots show a large number of points beyond the whiskers for almost every feature, confirming heavy-tailed distributions with many outlier emails that use a given word/character far more often than typical. Because MinMaxScaler (used in the scaling step) is sensitive to extreme values – a single very large outlier can compress the rest of the data into a narrow band near 0 – this is a known trade-off of the chosen scaling approach for this dataset.

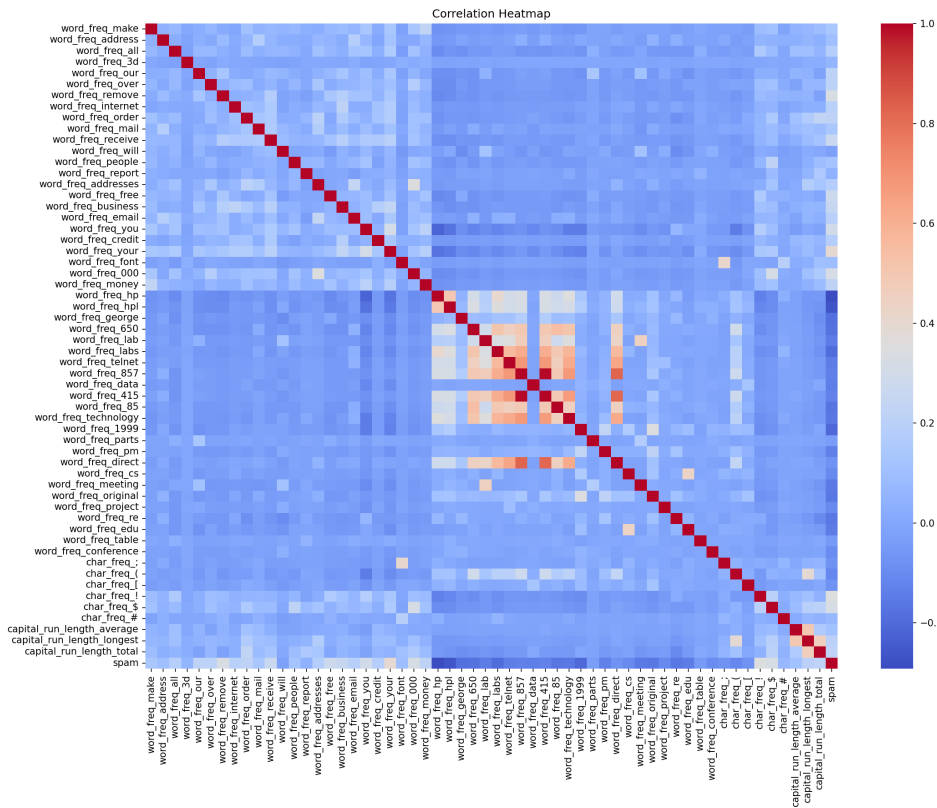


Figure 4: Correlation heatmap of all numeric features

Inference:

Most feature pairs show weak correlation with each other, meaning the dataset does not suffer from severe multicollinearity. A few clusters of moderately correlated features are visible among related word-frequency columns (e.g. business/technology- related terms), which is expected since certain vocabulary tends to co-occur within the same category of email.

5 Data Preprocessing

- **Missing value handling:** the single row with a missing target value was dropped (a row we can't train or test on). Remaining missing numeric feature values were filled with the column median (robust to the heavy-tailed distributions seen above).
- **Duplicates:** 384 exact duplicate rows were removed before splitting, so the model doesn't see the same email twice and to prevent duplicate rows from leaking between train and test.
- **Encoding:** not required – all 57 features are already numeric.
- **Feature scaling:** all features were scaled to the $[0, 1]$ range using MinMaxScaler, since the downstream algorithms in Experiment 2 (KNN, Naive Bayes) are sensitive to feature magnitude.
- **Train-test split:** 80/20 split, stratified on the target so both sets keep the same spam/ham ratio as the full dataset.

6 Machine Learning Algorithm

This experiment is a preprocessing/EDA pipeline rather than a specific learning algorithm; it prepares data for the classification algorithms evaluated in Experiment 2 (Naive Bayes and K-Nearest Neighbors). No model-specific working principle, mathematical formulation, advantages, or limitations apply at this stage.

7 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.9.0
IDE	Jupyter Notebook
Operating System	Windows
Hardware	Intel i5, 16GB RAM

8 Hyperparameter Selection

Not applicable for this experiment – no model is trained here. The only configurable parameters are pipeline-level: `test_size=0.20`, `random_state=42`, and the choice to scale with `MinMaxScaler`.

9 Results

Item	Value
Rows before cleaning	4,353
Duplicate rows removed	384
Rows with missing target dropped	1
Rows after cleaning	3,968
Features	57 (all numeric)
Training samples	3,174
Testing samples	794

Note: Accuracy/Precision/Recall/F1/ROC-AUC do not apply to this experiment – those are reported in the Experiment 2 report, where the actual classifiers are trained on the data prepared here.

10 Result Visualization

All figures are included above in the Exploratory Data Analysis section: class distribution, feature histograms, boxplots, and the correlation heatmap.

11 Discussion

The dataset required only light cleaning – a single missing target row and one missing value scattered across 24 other columns – but a substantial number of duplicate rows (384, roughly 9% of the raw data) needed to be removed to avoid train/test leakage and over-weighting repeated examples. The class distribution is mildly imbalanced (58:42), which was preserved across train/test via stratified splitting. Feature distributions are heavily right-skewed with many outliers, which is expected for word-frequency data and directly informs the choice of downstream classification algorithms in Experiment 2 (Multinomial Naive Bayes is theoretically better suited to this kind of count-like data than Gaussian Naive Bayes).

12 Conclusion

The generic preprocessing function successfully cleaned, scaled, and split the Spambase dataset with no data leakage, preserving class balance across the split. The resulting `X_train/X_test/y_train/y_test` were used directly as input to the classification experiments in Experiment 2.

13 References

1. Scikit-learn: Preprocessing data, <https://scikit-learn.org/stable/modules/preprocessing.html>
2. UCI Machine Learning Repository: Spambase Dataset